From click to code: what runs when you touch Meridian

This document traces every interaction on the Meridian page — every click, every keystroke in a box, every submitted form, every dialog — to the three places it lives:

Place	File	Job
The browser	`frontend/app.js`	listens for the interaction, calls the API, draws the answer
The API	`backend/app.py`	a Flask route matches the URL and calls one Python function
The work	`store.py`, `triage.py`, `analytics.py`, `registry.py`	read/write the CSV registries, rank symptoms, profile datasets

Each trace is written as a table row reading left to right:

what you do → which function fires in `app.js` → which URL is called → which function runs in `backend/app.py`.

The backend has not been edited since `docs/meridian-backend-code-explained.pdf` was written, so the two documents can be read side by side: that one explains the handler bodies line by line, this one explains how a handler gets called in the first place.

The round trip every interaction makes

Before reading the tables, it helps to see the full journey of one click (the Triage button) with names rather than words:

```
you press "Check what this is"
        |
triageForm.submit event               (the browser)
        |
function stored at "submit" listener  frontend/app.js
        |
api("/api/triage", { method: "POST", ... })
        |
fetch(..., Content-Type: application/json)     (the browser, again)
        |
POST /api/triage                       (HTTP, over the wire)
        |
@app.post("/api/triage")               backend/app.py
        |
def run_triage()
        |
triage.rank_diseases(...)  ->  candidates, warning lines
        |
jsonify({results...})                  (JSON back over the wire)
        |
renderTriage(data)                      frontend/app.js
        |
the answer appears on the page
```

Concepts in that chain that the lab practicals do not cover — `fetch`, promises with `async`/`await`, JSON bodies, the `<dialog>` element — each have a file in `docs/concepts/`, listed at the end of this document.

The only fetch: the `api()` helper

`app.js` contains exactly one place where a request leaves the page: the `api()` helper at the top of the file. Every trace below calls it.

```
async function api(path, options = {}) {
```

```
  response = await fetch(API + path, {
    headers: { "Content-Type": "application/json" },
    ...options,
  });
  ...
  payload = await response.json();
  if (!response.ok) {
    throw new Error(payload.error || `Request failed with HTTP ${response.status}.`);
  }
  return payload;
}
```

Three things worth noticing, because every view depends on them:

1. The interface never writes `fetch` anywhere else. One door in, one door out.
2. Errors arrive as JSON with an `error` message, and `api()` turns an `ok: false` response into a thrown error. That throw is caught by the calling view, which shows its own "Something went wrong / Retry" card — the error page never breaks the whole layout.
3. When the server cannot be reached at all (`fetch` rejects), the message becomes "Cannot reach the Meridian API. Is the server running?" — the sentence seen if the local server is off in a practical.

Opening the page and moving between views

You do	`app.js`	URL called	`app.py` function
Type the address, or press Enter	page load	`GET /`	`index()` (returns `index.html`)
Browser downloads the scripts	the `<script>` tags load	`GET /app.js`, `/app.css`, …	`frontend_assets(filename)`
Click a nav link (Triage, Patients …)	nothing yet: the link is `#patients`, so the browser fires `hashchange`	none	none
Tab key into the next view (`hashchange`)	`hashchange` listener calls `show(location.hash.slice(1) || "triage")`	none	none
An unknown hash such as `#nope`	`show()` normalises hyphens and falls back to `"triage"`	none	none

`show()` keeps the list `VIEWS` — `triage`, `patients`, `hospitals`, `doctors`, `medicines`, `stores`, `conditions`, `rare_conditions`, `analytics` — and reveals one `<section>` while hiding the rest. Navigation itself makes no request. Each section fetches the moment it is first shown, so the nearest row below ("what "a view being shown" does") carries the first real HTTP call for that view.

Triage: the form

The form in `index.html` has two fields, so the trace starts at one of two inputs:

You do	`app.js`	URL called	`app.py` function
Type symptoms or existing conditions	nothing yet — the field is just an input	none	none
Press "Check what this is" (`submit`)	the `submit` listener runs, disables the button, shows skeleton rows	`POST /api/triage`	`run_triage()`
(empty symptoms)	listener shows "No symptoms entered" and never sends	none	none
Answer arrives	`renderTriage(data)` paints the ranked results and the caution lines	—	—

Request fails	the catch shows the error card with a Retry button that calls `triageForm.requestSubmit()`	—	—

The body the browser sends is deliberately small — two plain strings:

```
{
  "symptoms": "fever, headache, body ache",
  "conditions": "asthma"
}
```

`run_triage()` reads them, checks something survived `normalise_symptoms` (that same function accepts lists, pipes and commas, which is why the interface can send the raw strings), then asks `triage.rank_diseases()` for candidates and builds the printable report. Two extra body keys — `age` and `patient_id` — are accepted by the backend for completeness, but the interface sends neither: the form has no age field, and no patient picker.

Registry views: typing a search, or browsing

Seven views — patients, hospitals, doctors, medicines, stores, conditions and rare conditions — are all built by the same `searchView()` factory, driven by two small config blocks in `app.js`:

Config block in `app.js`	Used when	Endpoints it names
`SEARCH_VIEWS`	you have typed something	`/api/search/patients`, `/api/search/hospitals`, `/api/search/doctors`, `/api/search/medicines`, `/api/search/stores`, `/api/search/diseases`, `/api/search/rare_conditions`
`BROWSE_VIEWS`	the box is empty and the list shows anyway	`/api/patients`, `/api/hospitals`, `/api/doctors`, `/api/medicines`, `/api/stores`, `/api/diseases` (shown as Conditions), `/api/rare-conditions`

You do	`app.js`	URL called	`app.py` function
Type in the search box	the `input` listener debounces 220 ms, then calls `load(query)`	`GET /api/search/<record_type>?q=…`	`search(record_type)` (route `/api/search/<record_type>`)
Clear the box	`load("")` swaps in the browse pager	`GET /api/<record_type>?page=&limit=…`	the `list_*()` for that registry
Click a facet chip, or "Show all"	`renderBrowse` re-fetches with that facet	same as above with facet flags	same handler, `request.args` filter
Press the older / newer page button	`onGo(±limit)` bumps `state.page`	same, next `page` value	same handler

Every list route funnels through the same helper chain in `app.py`: `page_args()` reads page and limit, `paged()` packs the current page with the true total, and — for patients — `attach_visit_counts()` adds the "seen N times" figure each row shows. The conditions browse additionally runs `attach_symptom_sets()`, which is why each row can preview the symptoms it explains.

A row click: the detail pages

Every row is a `<button>`, so clicking one fires a named `showX` function. Each one fetches exactly one URL, then builds the page.

You click	`app.js` function	URL called	`app.py` function

A patient row	`showPatient(p)`	`GET /api/patients/<id>`	`get_patient()`
A medicine row	`showMedicine(m)`	`GET /api/medicines/<id>`	`get_medicine()`
A store row	`showStore(s)`	`GET /api/stores/<id>`	`get_store()`
A hospital row	`showHospital(h)`	`GET /api/hospitals/<hospital_id>`	`get_hospital()`
A doctor row	`showDoctor(d)`	`GET /api/doctors/<id>`	`get_doctor()`
A condition row	`showCondition(id)`	`GET /api/diseases/<disease_id>`	`get_disease()` → `condition_detail()`
A rare-condition row	`showRare(c)`	`GET /api/rare-conditions/<condition_id>`	`get_rare_condition()`
The page back button, or Escape	`closeDetail()` (the box's `keydown` also cancels a pending search)	none	none

Two of these pages get more than a single row of the CSV, because the backend does the joining:

- `get_patient()` reads the patient, then walks `store.visits_for()`, and stamps each visit with its doctor and hospital so the page shows "who they saw and where" with no extra round trips.
- `get_disease()` calls `condition_detail()`, which assembles the whole article page in one go. Inside it, in order:
- `article_for()` supplies the written summary and the red-flag lines;
- `triage.rank_diseases()` is run with the condition's own symptom list as the query, which produces the "what else it could be" differentials;
- the article's medicine ids are looked up in the catalog and `registry.stores_with_medicine()` finds the pharmacies that stock them;
- every visit with that diagnosis is counted, giving "recorded here" and the last date seen.

Dialogs and one-off buttons

You do	`app.js`	URL called	`app.py` function
Rare view · press "How this is built"	the button's listener calls `loadRareOverview()` on first open, then `showModal()`	`GET /api/rare-conditions`	the rare list handler (the same call the browse uses; it carries the overview figures)
Rare overview · click a "most common finding" chip	the chip's onclick puts the word into the rare search box and runs `load()`	`GET /api/search/rare_conditions?q=…`	`search_records()`
Conditions view · press "How this is built"	the button opens `#conditions-about-dialog` with `showModal()` — the card is written in the HTML, so nothing is fetched	none	none
Header · press the sources button (the database icon next to the theme switch)	the button opens `#sources-dialog` with `showModal()` — the card and every link on it are written in the HTML	none	none
Any dialog · close button, backdrop click, or Escape	the `close()` listeners restore focus to the button that opened it	none	none

Theme button in the header	`theme.js` toggles a class and saves the choice in `localStorage`	none	none
The buddy button	`companion.js` shows and hides the mascot	none	none

The conditions dialog being static is worth one sentence: its card describes how the article pages are assembled, and everything it claims happens inside `condition_detail()` — verified numbers rather than a live count.

The sources card in the header is static for the same reason. It answers "where do these registries and datasets come from" with the five groups of provenance the research folder established — the CDC's ICD-10-CM download, the four open files behind the rare registry, what is real in the otherwise synthetic registries, the three UCI/Plotly cohort links, and what was written by hand. Each link on it was fetched and recorded before it was allowed onto the card.

The two write dialogs

The registries have been writable at the API all along, with validation, id numbering and their own tests. Two dialogs give that half a front:

You do	`app.js`	URL called	`app.py` function
Patients view · press "New patient"	the button's listener resets the form, opens the dialog, focuses the name field	none	none
Press "Register patient"	the submit handler packs the typed fields (date `registered_on` filled with today), disables the button	`POST /api/patients`	`create_patient()`
The answer arrives	dialog closes, the view re-runs the search on the new name, and a note reads "Registered P-0601 …"	`GET /api/search/patients?q=…`	`search()` (same door as typing)
Patient detail · press "Record a visit"	`openVisitForm(patient)` stamps the dialog with the patient's name and today's date	none	none
Press "Record visit"	submit includesthe invisible `patient_id`	`POST /api/visits`	`create_visit()`
The answer arrives	dialog closes, `showPatient()` reruns, and the new visit sits in the history	`GET /api/patients/<id>`	`get_patient()`

Nothing is validated in the browser. Whatever was typed travels to `registry.validate_record`, which answers with one list of field sentences; `api()` now carries that list on the thrown error, and the dialog paints it inside its own alert panel. A rejected field is therefore fixable without leaving the form.

Cohorts: the dataset pages

You do	`app.js`	URL called	`app.py` function
Switch to the Cohorts view for the first time	`show()` initialises it and calls `loadDatasets()`	`GET /api/analytics`	`list_analytics()`

Click a dataset tab	`selectDataset(name)` fires two requests together	`GET /api/analytics/<name>` and `GET /api/analytics/<name>/risk-bands`	`dataset_profile()` and `dataset_risk_bands()`
The risk-band answer arrives	`renderBands()` paints one line per banded column, with the unreachable bands noted	—	—
A band request fails	the `.catch(() => null)` in the same call keeps the profile page alive	—	—

`dataset_profile()` runs the pandas describe, groupby and value_counts for that dataset inside `analytics.profile()`, and `dataset_risk_bands()` maps column thresholds to counts inside `analytics.risk_bands()`. A dataset that never passes verification is turned away by `dataset_or_404()` with the list of usable names attached.

Build a graph

You do	`app.js`	URL called	`app.py` function
Open a dataset's builder	`getGraphCatalog()` runs once and caches	`GET /api/graphs`	`graph_catalog()`
Change the graph kind or first column	`syncOptions()` refills and hides the second column locally — no request	none	none
Press "Build graph"	the button's onclick builds `?x=…&y=…` and calls `api()`	`GET /api/graphs/<name>/<kind>`	`build_graph()`
The answer arrives	`addBuilt()` prepends a figure whose `<img src>` is `/charts/<file>`	`GET /charts/<file>`	`serve_chart()`

The dropdowns exist because `graph_catalog()` supplies the real kind list and the numeric columns of the chosen dataset; the browser never guesses a column name. A bad pair gets a 400 back whose message names what would have worked, which is how the form knows to say it.

One graph is thus two round trips: the request that renders the PNG and buys back its filename, then the image fetch that displays it.

Endpoints the interface never calls

Honesty row, useful when a reviewer asks "is this endpoint reachable?" All of the following exist in `backend/app.py` and are covered by the test suite, but no click or keystroke in `app.js` triggers them. They are API surface: for `curl` demonstrations in a practical, for the test suite, or for a write operation documented by the route.

Route	`app.py` function	Why it exists anyway
`PATCH /api/patients/<id>`	`update_patient()`	same; partial-update contract, curl
`DELETE /api/patients/<id>`	`delete_patient()`	the write half of the registry, exercised by tests and curl
`GET /api/visits`	`list_visits()`	visits reach the page pre-joined inside the patient detail
`POST /api/medicines`	`create_medicine()`	registry write, curl
`POST /api/stores`	`create_store()`	registry write, curl
`POST /api/hospitals`	`create_hospital()`	registry write, curl

`GET /api/diseases`	`list_diseases()`	yes — conditions browse
`GET /api/diseases/<id>`	`get_disease()`	yes — condition row click
`POST /api/triage`	`run_triage()`	yes — the triage form
`GET /api/analytics`	`list_analytics()`	yes — Cohorts first paint
`GET /api/analytics/risk-bands`	`list_risk_band_options()`	no
`GET /api/analytics/<name>`	`dataset_profile()`	yes — dataset tab click
`GET /api/analytics/<name>/risk-bands`	`dataset_risk_bands()`	yes — dataset tab click
`GET /api/analytics/<name>/charts`	`dataset_charts()`	no
`GET /charts/<file>`	`serve_chart()`	yes — every built graph's `<img>`
`GET /api/graphs`	`graph_catalog()`	yes — the builder's dropdowns
`GET /api/graphs/<name>/<kind>`	`build_graph()`	yes — Build a graph button
`POST /api/backup`	`run_backup()`	no

Concepts this walkthrough leans on

Where a concept is inside the lab syllabus it says the closest practical; where it is not, `docs/concepts/` has a dedicated explainer.

Concept	Where it appears	Closest lab or explainer
`fetch` and promises with `async`/`await`	`api()` in `app.js`, every view	requests.md, flask.md
JSON bodies and `Content-Type`	`api()`, `payload()` in `app.py`	json.md
Flask routing decorators and `jsonify`	every `@app.get` line in `app.py`	flask.md
`csv.DictReader` / `DictWriter`	every `store.py` call behind the routes	csv-module.md
Debounce with `setTimeout`/ `clearTimeout`	the search box input	plain JS timers, beyond syllabus
Event delegation and single listeners	row buttons, dialog close, pager	functions, Lab 3
The `<dialog>` element and `showModal()`	both "How this is built" cards	beyond syllabus, HTML5
`URLSearchParams` for query strings	the graph builder button	beyond syllabus
`matplotlib` figures, `savefig`	`render_graph()`, `save_charts()`	matplotlib-basics.md
pandas describe / groupby / value_counts	`profile()`, `risk_bands()`	Lab 6
`localStorage` and `matchMedia`	`theme.js`	beyond syllabus

Keeping this document true

Line numbers deliberately do not appear here — the mapping survives edits because it names functions, not offsets. Two rules keep it honest:

1. If a new `api()` call or list route is added, both tables in this document grow one row, in the same commit as the code.
2. Regenerate the PDF after any edit with:

```
python scripts/md_to_pdf.py
```